CLASSIFICATION OF VEHICULAR MOTION TRAJECTORIES IN A TRAFFIC VIDEO

Konapalli Kumar

CLASSIFICATION OF VEHICULAR MOTION TRAJECTORIES IN A TRAFFIC VIDEO

Thesis submitted to
Indian Institute of Technology Kharagpur
for the award of the degree

of

Master of Technology
in
Electrical Engineering
with specialization in
Signal Processing and Machine Learning

by

Konapalli Kumar
(23EE65R12)

Under the guidance of

Dr. Nirmalya Ghosh



DEPARTMENT OF ELECTRICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
APRIL 2025

© 2025, Konapalli Kumar. All rights reserved.

DECLARATION

I certify that

a. The work contained in the thesis is original and has been done by myself under the general supervision of my supervisor.

b. The work has not been submitted to any other Institute for any degree or diploma.

c. I have followed the guidelines provided by the Institute in writing the thesis.

d. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

e. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references.

f. Whenever I have quoted written materials from other sources, I have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Konapalli Kumar

Indian Institute of Technology Kharagpur
Department of Electrical Engineering
Kharagpur, WB 721 302, INDIA

CERTIFICATE

This is to certify that the thesis entitled **CLASSIFICATION OF VEHICULAR MOTION TRAJECTORIES IN A TRAFFIC VIDEO** submitted by **KONAPALLI KUMAR** to Indian Institute of Technology Kharagpur, is a record of bonafide research work under my supervision and I consider it worthy of consideration for the award of the degree of Master of Technology of the Institute.

Dr. Nirmalya Ghosh

Department of Electrical Engineering
Indian Institute of Technology Kharagpur

Date:
Place: Kharagpur, West Bengal, India

Phone: +91-3222-282-261, **Fax:** +91-3222-282-262, **url:** http://www.ee.iitkgp.ac.in/

Acknowledgments

I WOULD like to convey profound gratitude to **Dr. Nirmalya Ghosh**, for his invaluable guidance, insightful feedback, and consistent support throughout the duration of my thesis. His extensive knowledge, academic rigor, and mentorship have played a pivotal role in shaping the quality and direction of this work. It has been a privilege to learn under his supervision, and I remain deeply appreciative of the time and effort he has devoted to my academic development. I would also like to express my sincere thanks to **Prof. Debdoot Sheet** for his instrumental role in supporting the practical aspects of this thesis. His insightful suggestions, technical ideas, and guidance on working with the Quanser QCar platform were crucial in translating my concepts into real-world experimentation. I am grateful for the access he provided to the QCar laboratory, the necessary hardware components, and the cam- era equipment used for data collection. His support significantly enhanced the scope and practical relevance of my research.

I am sincerely thankful to my family, whose unwavering support and encouragement have been fundamental to the successful completion of my M.Tech journey. I extend my heartfelt appreciation to my father, **Mr. Konapalli Varahalu**, and my mother, **Mrs. Konapalli Ramalakshmi**, for their unconditional love, patience, and the immense freedom they afforded me during my studies. Their willingness to relieve me of other responsibilities during this crucial period enabled me to focus fully on my academic and research pursuits. I also wish to express my genuine appreciation to my brother, **Mr. K.V.S. Kishore**, for his constant motivation and quiet encouragement. His faith in my capabilities and his steadfast support have been an invaluable source of strength throughout this journey. I sincerely thank all who contributed, directly or indirectly, to this thesis—especially fellow researchers, seniors, and friends—for their guidance, encouragement, and support throughout this journey

This thesis stands as a testament to the collective support, sacrifice, and belief of my mentors, family, and friends. I dedicate this work to all of them with sincere gratitude and affection.

Contents

List of Figures ix

List of Tables xi

List of Abbreviations xii

List of Symbols xiii

1 Introduction 1

1.1 Aim and Objectives 3

1.2 Outline of Thesis 4

2 Literature Review 5

2.1 Traditional Vehicle Detection Methods 5

2.2 U-Net Based Segmentation 6

2.3 YOLO-Based Approaches 6

2.4 Challenges in Vehicle Detection and Tracking 7

2.5 Gaps in Existing Work 7

3 Vehicle Detection and Tracking 9

3.1 Vehicle Detection 9

3.1.1 Detection and tracking methodology 9

3.1.2 Problems encountered 10

3.1.3 Proposed methodology for enhanced vehicle detection 10
3.1.4 Enhanced vehicle detection 13
3.1.5 Binary masking and contour extraction 14
3.1.6 Centroid calculation 15
3.2 Vehicle Tracking 17

4 Trajectory fitting and Data Generation 20
4.1 Polynomial Fitting 20
4.1.1 Vehicle centroid representation 21
4.1.2 Polynomial points for curve fitting 21
4.1.3 Sub-trajectory fitting for accurate path modeling 25
4.2 Data Generation 26
4.2.1 Data imbalance in trajectory classes 26
4.2.2 Data augmentation 27
4.3 Summary 30

5 Trajectory Classification using LSTMs 31
5.1 LSTM-Based Classifier Model 31
5.2 LSTM Auto-Encoder 33
5.2.1 Encoding layer 35
5.2.2 Decoding layer 35
5.3 Trajectory classification 36
5.3.1 Training with normal and partial-length trajectories 36
5.3.2 Testing with full-length and abnormal trajectories 37
5.3.3 Trajectory classification using LSTM-based autoencoder 37
5.3.4 Model evaluation and abnormality detection 38
5.4 Results and Observations 38
5.5 Summary 40

6 Vehicle Speed-Based Classification 42
6.1 Introduction 42

6.2 Pixel-wise Speed Profiling Strategy 42
6.2.1 Trajectory pixel mapping 42
6.2.2 Neighborhood-based speed aggregation 43
6.3 Vehicle Speed Calculation 44
6.3.1 Centroid-based speed estimation 44
6.4 Speed-Based Anomaly Detection 44
6.4.1 Statistical thresholding 44
6.4.2 Advantages of local speed thresholding 45
6.5 Summary 45

7 Conclusion **46**
7.1 Summary of the Work 46
7.2 Future Work 46

Related Publications **48**

References **49**

Curriculum vitae **50**

List of Figures

3.1 Vehicle detection using movable kernel-based centroid fixing. Row 1 shows original frames, Row 2 shows YOLOv8 detections (blue), and Row 3 highlights vehicles detected by kernel-based centroid fixing (red). 12

3.2 Vehicle detection with masks using Mask R-CNN (The highlighted parts with black color are non-vehicular parts). 13

3.3 Vehicle segmentation using Mask R-CNN after eliminating the non vehicular parts 14

3.4 Binary mask generation from Mask R-CNN for vehicle segmentation. . 15

3.5 Contour extraction from binary masks to delineate vehicle boundaries. . 15

3.6 Centroid refinement using image moments for accurate vehicle localization. 17

3.7 Vehicle tracking using centroid distance-based vehicle labeling. 18

4.1 Fitted curves(in Blue) along the centroid paths (in Red) of different vehicles. 25

4.2 Images depicting curve fitting considering the entire curve as a single trajectory (a), curve fitting using sub-trajectories (b), and the sub-trajectories curve fitting shown in 20 different segments of the trajectory (c). 25

4.3 Normal trajectories of a few vehicles captured in different colors 28

4.4 Generated normal trajectories (in Blue) compared to the already encountered sample trajectory (in Red). 29
4.5 Generated abnormal trajectories 29

5.1 Architecture of LSTM-based Auto-Encoder and Decoder 36
5.2 In Figure 5.2b, the orange trajectory represents a normal trajectory with a path that overlaps a known abnormal path. Similarly, in Figure 5.2a, the red normal trajectory shows considerable overlap with abnormal trajectories. 40

6.1 (a) Binary mask of pixels traversed by vehicles. (b) A local 15-pixel neighborhood used to aggregate speed information around trajectory points. 43
6.2 Speed distribution aggregated over the 15-pixel neighborhood around multiple trajectory pixels. The horizontal axis indicates speed values, while the vertical axis indicates frequency. Peaks reflect commonly observed speeds in that local region. 43

7.1 Q-Car setup 47

List of Tables

5.1 Comparison of metrics for full length and 70% length of trajectories . . 40

List of Abbreviations

YOLO	You Only Look Once
R-CNN	Region-based Convolutional Neural Network
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
ROI	Region of Interest
COCO	Common Objects in Context
PDF	Probability Density Function
MSE	Mean Squared Error
FPS	Frames Per Second
RPN	Region Proposal Network

List of Symbols

C_x, C_y	Centroid coordinates of the vehicle
M_{00}, M_{10}, M_{01}	Image moments used for centroid calculation
d	Euclidean distance between centroids
v	Vehicle speed
t	Time (frame index or duration)
x, y	Pixel or spatial coordinates
w	Polynomial regression weight vector
X^T	Transposed design matrix for polynomial fitting
A_{ij}	Elements of the Gram matrix in polynomial regression
μ	Mean speed in statistical thresholding
σ	Standard deviation of speed
$\hat{x}_t$	Reconstructed point in trajectory by decoder
ε	Noise for data augmentation
$f(x)$	Polynomial curve function
$E(w)$	Error function in polynomial fitting
$i_t, f_t, o_t, \tilde{c}_t, h_t$	Input, forget, output gates, candidate cell state, and hidden state in LSTM
N	Number of samples or features

THis thesis presents a comprehensive framework for accurate vehicle detection, consistent ID tracking, motion trajectory classification, and speed-based anomaly detection in traffic video sequences. The proposed system integrates the real-time efficiency of You Only Look Once Version 8 (YOLOv8) for initial vehicle detection with the precise segmentation capabilities of Mask Region-based Convolutional Neural Network (Mask R-CNN) to enhance detection accuracy, especially in complex traffic scenarios. To address potential missed detections due to low confidence scores or scene complexities, a movable kernel-based enhancement is incorporated, ensuring more comprehensive coverage around each bounding box. For tracking and ID consistency, a centroid distance-based labeling method is employed, where bounding box centroids are calculated and compared frame-to-frame using Euclidean distance, maintaining consistent vehicle identities across the consecutive frames. To further refine detection and centroid localization, particularly in occluded scenes, Mask R-CNN is utilized to segment vehicles and extract precise contours, facilitating accurate centroid computation. These centroids are then used to generate smooth motion trajectories through polynomial fitting. Sub-trajectories extracted from these paths are analyzed using a Long Short-Term Memory (LSTM) network to classify vehicle trajectory patterns as normal or abnormal. Beyond trajectory analysis, the framework also estimates vehicle speeds by calculating the displacement of centroids across consecutive frames and then compared with the distribution of speeds within defined Regions of Interest (ROIs). By fitting a normal distribution to the distribution of these speeds, the system classifies vehicular movements as normal or abnormal based on deviations from the mean speed, enhancing the capability for intelligent traffic monitoring and anomaly detection as well as prediction (for alarming neighboring vehicles) ahead of time. Overall, this integrated approach offers a robust and scalable solution for real-world traffic analysis, combining high detection accuracy, consistent tracking, and intelligent classification of vehicular motion and speed anomalies in dynamic and real traffic environments.

Keywords: Dynamic motion tracking, speed abnormalities, traffic monitoring, trajectory classification, vehicle detection.

1 Introduction

Accurate detection and tracking of vehicles are fundamental components of traffic monitoring systems, essential for analyzing vehicular motion trajectories (Morris & Trivedi 2008). These analyses play a crucial role in various applications, including traffic management, urban planning, and the development of autonomous driving technologies. However, despite the advancements in deep learning models like YOLOv8 (Afdhal et al. 2023), challenges persist that can undermine the reliability of these systems. YOLOv8, known for its efficiency in real-time object detection, often struggles with complex traffic environments where factors such as occlusion, varying lighting conditions, and the presence of small or distant objects complicate detection efforts. These challenges can result in vehicles being intermittently missed or inaccurately tracked across the frames, leading to gaps in tracking results and inconsistencies in trajectory analysis.

The motivation behind this work is to address these limitations by developing a robust methodology that ensures consistent detection and accurate tracking of all vehicles, even in challenging traffic scenarios. By enhancing the detection accuracy and improving the consistency of tracking, the goal is to significantly boost the reliability and precision of vehicular motion trajectory classification (Zeng et al. 2021). This, in turn, will support critical applications in traffic management (Leilei et al. 2012), safety analysis, and the burgeoning field of autonomous vehicles. The proposed methods aim to overcome the shortcomings of existing models, thereby contributing to more dependable and effective traffic monitoring systems.

Significance

1. **Enhancing Traffic Management and Surveillance**:

   Effective traffic management requires a comprehensive understanding of vehicle trajectories. By analyzing trajectories extracted from video footage, transportation authorities can gain valuable insights into traffic patterns, congestion hotspots, and network performance. These insights contribute to reduced travel times, improved fuel efficiency, and overall enhancements in transportation system efficiency.

   Furthermore, vehicle trajectory classification plays a crucial role in surveillance and security systems. By monitoring vehicle movements through video feeds, authorities can identify abnormal or suspicious behaviors, such as unauthorized entries or erratic driving patterns. This capability supports timely interventions and strengthens the security of sensitive areas, critical infrastructure, and public spaces.

2. **Improving Safety and Operational Efficiency**:

   Understanding vehicle trajectories is vital for accident prevention, incident response, and data-driven decision-making in transportation policy and infrastructure planning. Trajectory classification helps identify hazardous driving behaviors, anticipate potential collisions, and implement proactive safety measures. Additionally, it supports post-accident investigations by providing insight into vehicle movement patterns and possible causes.

   Trajectory data also informs urban planning efforts. By studying traffic flows and movement patterns, city planners can make informed decisions about road design, traffic signal placement, and public transit routing. This leads to safer and more efficient urban environments that accommodate the needs of residents and commuters. The insights derived from classified trajectories enable policymakers to track trends, evaluate the effectiveness of interventions, and develop evidence-based strategies to address issues such as congestion, pollution, and road safety.

In this thesis, the focus is on refining techniques for vehicle detection and trajectory analysis using static traffic video footage. The approach begins with vehicle detection using YOLOv8 (Bawankule et al. 2023), followed by centroid-based tracking to improve detection consistency. To overcome the limitations of single polynomial fitting, a novel sub-trajectory fitting method is introduced. This technique enables a more detailed and accurate representation of vehicle movements, which in turn enhances the effectiveness of trajectory analysis and classification.

1.1 Aim and Objectives

The Aim of this current study is to develop a robust framework for vehicle detection, tracking, and trajectory-based classification in traffic videos, enabling accurate centroid extraction, consistent tracking, and identification of abnormal vehicle movements based on trajectory patterns and vehicle speed classification as normal or abnormal.

The specific objectives of this work are:

1. Detect vehicles using YOLOv8 and overcome bounding box limitations through enhanced detection techniques.

2. Implement Mask R-CNN for pixel-level vehicle detection to handle occlusion, overlapping, and complex traffic conditions. Extract accurate centroids from binary masks generated by Mask R-CNN.

3. Track vehicles consistently across frames using a centroid-based ID assignment method with Euclidean distance.

4. Generate smooth vehicle trajectories using polynomial fitting at sub-trajectory levels (taking segments of complete trajectories) for motion analysis.

5. Classify vehicular motion trajectories as normal or abnormal using an LSTM-based deep learning model.

6. Classify vehicle speeds as normal or abnormal based on statistical thresholds derived from instantaneous speed distribution in the neighborhood from the trajectory data.

1.2 Outline of Thesis

Chapter 1 presents the Introduction to the research problem, along with the aim and specific objectives that guide the overall study. Chapter 2 describes the literature review performed on various existing trajectory classification methods.

Chapter 3 contains methodologies implemented for vehicle detection and tracking.

Chapter 4 discusses polynomial regression for curve fitting and sub trajectories generation. Data generation using sub trajectories included in this chapter.

Chapter 5 explains LSTM based auto-encoder model for classifying the trajectories. The training process, testing procedures, and corresponding results have been discussed.

Chapter 6 discusses the classification of vehicles speeds as normal or abnormal. Finally, Chapter 7 summarizes the work with concluding remarks and future research directions.

2 Literature Review

The classification of vehicular motion trajectories has been explored through various methodologies, each with its own advancements and limitations. This chapter presents an overview of the key detection and tracking approaches explored in the literature, highlighting their performance in different traffic surveillance contexts.

2.1 Traditional Vehicle Detection Methods

Initially, vehicle detection relied heavily on traditional methods such as background subtraction (Sakpal & Sabnis 2018). This technique compared each video frame with a reference background model to identify moving vehicles as foreground objects. Histogram-based approaches, for instance, computed pixel-wise intensity distributions over time to generate a static background image. Foreground regions were then extracted by subtracting this background from the current frame. While computationally simple, these methods face significant challenges. Shadows moving at the same speed as vehicles are often detected as part of the foreground (Tiwari et al. 2016), leading to false positives. Furthermore, vehicles with low-intensity values or those positioned far from the camera often remain undetected due to poor contrast with the road surface. These issues lead to inaccurate segmentation and fragmented tracking, especially in real-world traffic conditions involving clutter and varying lighting.

2.2 U-Net Based Segmentation

To address the limitations of classical background subtraction methods, deep learning-based semantic segmentation models such as UNET were introduced. The UNET model (Hussain et al. 2022), trained on large datasets, aimed to improve the precision of vehicle detection by learning pixel-level distinctions between vehicles and the background. Despite its advantages, UNET struggled in practical traffic scenes. It frequently segmented multiple adjacent vehicles as a single region and often misclassified vehicle parts, such as shadows or roofs, as separate objects. This caused incorrect vehicle labeling and significantly hampered tracking continuity across frames. Moreover, the model's inference speed and adaptability to real-time traffic footage were limited, reducing its viability for deployment in real-time surveillance systems.

2.3 YOLO-Based Approaches

The introduction of the YOLO (You Only Look Once) series, particularly YOLOv8 (Redmon et al. 2016), represented a major step forward in vehicle detection. YOLOv8 utilizes a single-pass deep convolutional neural network that partitions the image into a grid and predicts both bounding boxes and class probabilities for each grid cell. Its real-time performance, combined with high detection accuracy, makes it well-suited for dynamic environments such as road traffic monitoring. YOLOv8 offers several key advantages such as Robust detection even under partial occlusions or clutter, Significant reduction in false positives and mis-identifications compared to UNET and Generalizability across varied lighting conditions and camera perspectives due to extensive pre-training. However, YOLOv8 is not without limitations. Occasionally, vehicles are missed due to low confidence scores, especially when they are far from the camera or partially visible. The bounding boxes may shift across frames, causing centroid inconsistencies and identity switches during tracking. These shortcomings limit its standalone utility in high-accuracy tracking applications.

2.4 Challenges in Vehicle Detection and Tracking

Across all methods, several persistent challenges affect the accuracy and robustness of vehicle trajectory analysis:

- **Occlusion:** Vehicles may be temporarily hidden behind others, resulting in missed detections or misclassifications.
- **Shadow Confusion:** Moving shadows are frequently misinterpreted as vehicle parts, leading to segmentation errors.
- **Low Confidence Detections:** Small or distant vehicles may not reach the confidence threshold of detection models like YOLOv8.
- **ID Switching:** Inconsistent centroid estimation across frames results in broken vehicle trajectories or reassigned IDs.
- **Bounding Box Limitations:** The centroid calculated from a bounding box does not always align with the physical center of the vehicle, affecting trajectory accuracy.

These issues call for a more integrated solution that combines object detection with fine-grained segmentation and robust tracking.

2.5 Gaps in Existing Work

While traditional and deep learning-based methods have made significant contributions, there are critical gaps that remain unaddressed:

- Most frameworks rely on bounding boxes alone for centroid estimation, which introduces errors in trajectory generation.
- Few approaches consider the recovery of missed detections due to low confidence, especially in dynamic traffic scenes.

- Temporal consistency of object IDs across frames is often not enforced, leading to mislabeling and fragmented trajectories.
- Classification of motion trajectories is often performed on raw coordinates, which are sensitive to scale and noise.

To address these limitations, the present work integrates YOLOv8-based detection with a movable kernel enhancement, applies Mask R-CNN for pixel-accurate vehicle segmentation, and employs polynomial trajectory fitting with LSTM-based classification to distinguish normal and abnormal vehicular motion. This hybrid approach ensures improved tracking consistency and higher accuracy in motion classification.

3 Vehicle Detection and Tracking

3.1 Vehicle Detection

The initial approach in this study involved using the YOLOv8 model for vehicle detection and tracking across video frames. However, challenges arose when YOLOv8 failed to detect certain vehicles, leading to missing bounding boxes and incorrect tracking IDs.

3.1.1 Detection and tracking methodology

The methodology employed several key steps to ensure accurate vehicle detection and consistent tracking:

- **YOLOv8 for Vehicle Detection:** YOLOv8, known for its real-time capabilities, was used to detect vehicles in each frame. The model's robustness in handling diverse traffic conditions made it suitable for this study. Vehicle detection was determined by evaluating the area of the bounding boxes generated by YOLOv8. If the area was less than or equal to 300 pixels (the threshold value), the object was considered a vehicle. However, detection failures occurred due to factors like occlusions, changes in vehicle size, and low contrast.

- **Challenges with YOLOv8 and Gaps in Data:** YOLOv8's failure to detect vehicles in some frames resulted in missing centroids, disrupting trajectory continuity and hindering accurate tracking.

3.1.2 Problems encountered

Several challenges were identified during implementation:

- **Bounding Box Area Threshold and Tracking Challenges:** A major issue was that if the area of a detected vehicle's bounding box exceeded the threshold of 300 pixels, the system failed to recognize it as a vehicle. This led to certain vehicles being undetected, impacting overall detection accuracy. Additionally, in previous work, there was no mechanism to track vehicles consistently from frame to frame. Consequently, the system struggled to maintain the continuity of vehicle identities across frames, leading to inconsistencies in vehicle association.

- **Low Confidence Scores and Detection Failures:** Another significant problem was YOLOv8's failure to detect vehicles with low confidence scores(less than 0.5 threshold value), resulting in missing bounding boxes for some vehicles. This led to incomplete detection, where certain vehicles were not shown with bounding boxes, disrupting the accuracy and completeness of the tracking process.

3.1.3 Proposed methodology for enhanced vehicle detection

This section introduces an enhanced approach to improve vehicle detection and tracking accuracy, particularly in scenarios where standard methods like YOLOv8 encounter challenges. The key components of this approach are a centroid-based movable 5×5 neighborhood search for vehicle detection, the use of low-confidence object detection through the YOLOv8 model, and a centroid distance-based vehicle labeling method.

Centroid-based movable 5×5 neighborhood search for vehicle detection

The proposed method involves dynamically moving a 5×5 grid, referred to as the kernel, within and around the known bounding box of a vehicle in each frame. This approach differs from a fixed centroid method as it allows the kernel to explore potential positions within the bounding box, thereby accommodating shifts in the vehicle's

location due to predictable or unpredictable movement or partial occlusion. The process begins with the detection of a vehicle and the establishment of its bounding box in a given frame. The kernel is then systematically moved within the boundaries of this bounding box. As the kernel moves, the YOLO model is employed to check for objects that may have been detected with low confidence but not classified as vehicles. These objects, although detected with low confidence, are reconsidered in this step to improve tracking accuracy.

Algorithm 1 Kernel Movement for Vehicle Detection and Tracking

Require: Initial centroid $C_{\text{current}} = (x_c, y_c)$, Bounding box B, YOLO model
Ensure: Updated centroid C_{new}

1: **Initialization:** Place the kernel at C_{current}
2: **Step 1:** Initial Position: Place the kernel at the centroid of the bounding box C_{current}
3: **Step 2:** Movement Path: Move the kernel systematically within a local 5×5 pixel neighborhood around bounding box B
4: **for** each kernel position k within the bounding box B
5: **Step 3:** YOLO Query: Query the YOLO model to detect low-confidence objects at position k
6: Record detected centroids $C_k = (x_k, y_k)$ from YOLO detections by entire region covered by kernel
7: **Step 4:** Distance Calculation: Compute the Euclidean distance d_k between C_{current} and each detected centroid C_k as:

$$d_k = \sqrt{(x_c - x_k)^2 + (y_c - y_k)^2}$$

8: Store the distances d_k for all detected centroids C_k
9: **end for**
10: **Step 5:** Centroid Update: Identify the minimum distance $d_{\min} = \min(d_k)$
11: Set C_{new} as the centroid corresponding to $d_{\min}$:

$$C_{\text{new}} = [x_m, y_m] \quad \text{where} \quad m = \arg\min_k (d_k)$$

12: Update the vehicle's centroid to C_{new}

At each position of the kernel within the bounding box, the YOLO model is queried for low-confidence detections. The bounding boxes of these low-confidence scores(below 0.5 threshold)objects are then analyzed, focusing on their centroid positions. To determine the vehicle's most likely centroid, the Euclidean distance between the known bounding box centroid and the centroids of the low-confidence objects detected by the YOLO model is calculated. The Euclidean distance d_k between the known centroid

$C_{\text{current}} = (x_c, y_c)$ and each detected centroid $C_k = (x_k, y_k)$ for the low-confidence objects is given by the following equation:

$$d_k = \text{dist}(C_{\text{current}} - C_k) = \sqrt{(x_c - x_k)^2 + (y_c - y_k)^2} \tag{3.1}$$

Here, $k = 1, 2, 3, \ldots, n$ represents the indices of the detected objects. Among all these distances, the centroid C_{new} with the minimum distance is selected as the vehicle's probable new centroid, updating its position accordingly. The centroid with the minimum distance is determined using the equation:

$$C_{\text{new}} = \arg\min_{C_k} \text{dist}(C_{\text{current}} - C_k) \tag{3.2}$$

In this method, the kernel moves systematically within a local 5×5 pixel neighborhood around the bounding box B, ensuring that all potential positions are explored. The kernel shifts in small increments, covering both the interior and boundaries of the bounding box. This comprehensive movement helps account for slight changes in the vehicle's position that might not be captured by a static approach.



Figure 3.1: Vehicle detection using movable kernel-based centroid fixing. Row 1 shows original frames, Row 2 shows YOLOv8 detections (blue), and Row 3 highlights vehicles detected by kernel-based centroid fixing (red).

3.1.4 Enhanced vehicle detection

Mask R-CNN (Wu et al. 2020) is a versatile framework for object detection and segmentation, capable of identifying and segmenting objects at a pixel level. This method combines a ResNet-50 backbone for feature extraction, a Region Proposal Network (RPN) for object localization, and a segmentation branch for boundary detection. These components enable Mask R-CNN to effectively segment vehicles, even in cases of partial occlusion or irregular shapes. Initially, Mask R-CNN detected vehicles successfully shown in Figure3 but also misclassified non-vehicular objects such as traffic poles and parts of buildings as vehicles. This misclassification occurred due to overlapping visual features between non-vehicle objects and vehicles, as well as the limitations of the pre-trained model in distinguishing such cases. For example, narrow and vertical traffic poles were misclassified as vehicle parts, and flat building surfaces were mistaken for trucks or buses.



Figure 3.2: Vehicle detection with masks using Mask R-CNN (The highlighted parts with black color are non-vehicular parts).

Additionally, a relatively low confidence threshold allowed false positives, where objects with lower detection confidence were included. To address these issues, the methodology was refined by filtering detections to include only vehicles (cars, buses, and trucks) based on their predefined Common Objects in Context(COCO) class IDs. The confidence threshold was increased to eliminate low-confidence detections, and segmentation was refined to ensure that only vehicle-related masks were overlaid onto

the original frame. These improvements produced clean and accurate segmentation shown in figure 4, removing non-vehicle objects and focusing exclusively on vehicles.



Figure 3.3: Vehicle segmentation using Mask R-CNN after eliminating the non vehicular parts

3.1.5 Binary masking and contour extraction

Binary masking is a critical preprocessing step in object segmentation, where an image is converted into a black-and-white representation to highlight specific regions of interest. In this work, Mask R-CNN generates pixel-level segmentation maps, which are transformed into binary masks by isolating regions corresponding to vehicles such as cars, buses, and trucks. Pixels within the vehicle regions are set to white (foreground), while all other pixels are set to black (background). These binary masks shown in figure 5 simplify the representation, ensuring only vehicle-specific regions are retained for further processing.



Figure 3.4: Binary mask generation from Mask R-CNN for vehicle segmentation.

Once the binary masks are generated, contours are extracted to identify the exact boundaries of the vehicles. Contours provide precise geometric information about the shape and position of the vehicles, enabling accurate centroid calculation, which is essential for motion tracking and trajectory analysis. This methodology ensures robust localization and boundary detection of vehicles, forming a reliable foundation for analyzing their movements and behaviors in complex traffic scenes.



Figure 3.5: Contour extraction from binary masks to delineate vehicle boundaries.

3.1.6 Centroid calculation

The centroid of a detected vehicle represents its geometric center or center of mass, providing a reliable reference point for tracking its motion across frames. In this work,

centroids are calculated using image moments derived from the binary masks of vehicles. Image moments quantify the distribution of pixel intensities within a region and are used to calculate the centroid coordinates (C_x, C_y) as follows:

$$C_x = \frac{M_{10}}{M_{00}}, \quad C_y = \frac{M_{01}}{M_{00}} \tag{3.3}$$

Here:

- M_{00}: Zeroth-order moment, representing the total area of the contour. It is calculated as:

$$M_{00} = \sum_x \sum_y I(x,y) \tag{3.4}$$

  where $I(x,y)$ is the pixel intensity at position (x,y). For binary images, $I(x,y) = 1$ for foreground pixels (vehicle region) and $I(x,y) = 0$ for background pixels.

- M_{10}: First-order moment along the x-axis, calculated as:

$$M_{10} = \sum_x \sum_y x \cdot I(x,y) \tag{3.5}$$

- M_{01}: First-order moment along the y-axis, calculated as:

$$M_{01} = \sum_x \sum_y y \cdot I(x,y) \tag{3.6}$$

The general formula for calculating the (p,q)-order moment is given by:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x,y) \tag{3.7}$$

Where:

- p,q: Non-negative integers representing the order of the moments.

- $I(x,y)$: Pixel intensity at position (x,y).

- x,y: Pixel coordinates in the binary mask.

The binary mask, generated from the Mask R-CNN output, highlights the vehicle's region in white while keeping the background black. Contours are extracted from these binary masks to delineate the boundaries of the vehicle. Using these contours, the image moments are computed, and the centroid coordinates are derived using Equation 3.3.

This method ensures the centroid represents the true geometric center of the vehicle's boundary. Unlike bounding box-based centroid approximations, this moment-based calculation is significantly more accurate as it adapts to the actual shape of the vehicle. The figure below illustrates the refinement of centroids from bounding box-based methods to moment-based centroids:



Figure 3.6: Centroid refinement using image moments for accurate vehicle localization.

3.2 Vehicle Tracking

Centroid distance-based vehicle labeling with correct ID

To enhance the tracking accuracy and consistency of vehicle IDs across frames, a centroid distance-based labeling method is implemented. This approach ensures that vehicles maintain consistent IDs throughout the video sequence by matching bounding boxes in consecutive frames based on the Euclidean distance between their centroids. As vehicles move through the video, the centroids of their bounding boxes will naturally shift from one frame to the next. The centroid $C(x,y)$ of each bounding box is calculated as follows:

$$C(x,y) = \left(\frac{x_{\min} + x_{\max}}{2}, \frac{y_{\min} + y_{\max}}{2} \right) \tag{3.8}$$

where $x_{\min}$, $x_{\max}$, $y_{\min}$, and $y_{\max}$ are the coordinates of the bounding box corners.

To determine whether a vehicle detected in the current frame corresponds to the same vehicle detected in the previous frame, the Euclidean distance d between the centroids $C_1(x_1, y_1)$ in the previous frame and $C_2(x_2, y_2)$ in the current frame is calculated as follows:

$$d(C_1, C_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{3.9}$$

A threshold distance $d_{\text{threshold}} = 35$ pixels is set to make this determination. If the distance $d(C_1, C_2)$ between the centroids of the bounding boxes in the current and previous frames is less than or equal to 35 pixels, the bounding boxes are considered to represent the same vehicle, and the same ID is assigned to the vehicle in both frames. If the distance exceeds 35 pixels, the bounding box in the current frame is considered to belong to a new vehicle, and a new ID is assigned. This centroid distance-based labeling method effectively tracks vehicles as they move across frames, ensuring consistent vehicle IDs. It also distinguishes new vehicles entering the scene, maintaining accurate tracking throughout the video sequence that follows from thereon.



Figure 3.7: Vehicle tracking using centroid distance-based vehicle labeling.

This new approach was compared with the initial YOLO-based method. It was

observed that this enhanced method significantly improved detection accuracy and consistency in tracking, particularly in cases where YOLO failed due to low confidence or bounding box shifts. The results of this enhanced methodology showed that vehicle detection and tracking were more consistent, even in challenging scenarios where vehicles were partially occluded or detected with low confidence.

4 Trajectory fitting and Data Generation

4.1 Polynomial Fitting

Linear regression is a fundamental statistical technique used to model the relationship between two variables by fitting a straight line to a set of data points. The objective is to find the "best fit line" that minimizes the overall distance between the observed data points and the line itself. This method works well when the relationship between the variables is linear, meaning that the data points tend to form a roughly straight line when plotted on a graph. However, in scenarios where the data exhibits non-linear patterns, such as trajectory paths of vehicles, linear regression may not be suitable for accurately capturing the underlying relationships. Trajectory paths of vehicles are inherently dynamic and complex, influenced by various factors such as acceleration, deceleration, changes in direction, and environmental conditions. These factors result in non-linear behavior, where the trajectory path does not follow a simple straight line. In such cases, using linear regression to model the trajectory path would likely result in a poor fit and inaccurate predictions. Polynomial regression offers a more flexible approach for modeling non-linear relationships in data. Instead of fitting a straight line, polynomial regression fits a curve to the data, allowing for greater flexibility in capturing complex patterns. By employing polynomials of different degrees, polynomial regression can accommodate a wide range of non-linear relationships, providing a better fit to the data compared to linear regression. In the context of vehicle trajectory analysis, polynomial regression is particularly useful for accurately modeling the non-linear behavior

of trajectory paths. By fitting a higher-order polynomial curve to the trajectory data, polynomial regression can capture the intricacies of vehicle movement.

4.1.1 Vehicle centroid representation

The centroids of the vehicle serve as pivotal markers along the trajectory path of a specific vehicle. Each centroid is characterized by a pair of coordinates (t_i, x_i), where t_i denotes the target y-coordinate and x_i represents the observation x-coordinate. These centroids are derived from the binary mask that outlines the precise vehicle region, rather than from bounding boxes. This approach provides more accurate and reliable spatial data points for subsequent trajectory analysis.

4.1.2 Polynomial points for curve fitting

In the context of curve fitting to represent the vehicle's trajectory, the provided centroids $(t_1, x_1), (t_2, x_2), \ldots, (t_N, x_N)$ act as the training dataset. Here, $x_1, x_2, \ldots, x_N$ are regarded as the observations, signifying the x-coordinates of the centroids, while $t_1, t_2, \ldots, t_N$ represent the corresponding target values, representing the y-coordinates. This distinction is fundamental for accurately capturing the relationship between the vehicle's trajectory in terms of its x and y coordinates.

Representation of vehicle centroid values

To efficiently handle and process this dataset, a vector representation proves invaluable. The observations vector contains the x-coordinates of the centroids, denoted as $x_1, x_2, \ldots, x_N$. Simultaneously, the target values vector contains the y-coordinates of the centroids, represented by $t_1, t_2, \ldots, t_N$. By organizing the data in this structured manner, it becomes easier to apply curve fitting algorithms like polynomial regression, allowing us to effectively model the non-linear relationship between the x and y coordinates of the centroids. We can represent these data points in vector form as follows:

$$\mathbf{t} = \begin{bmatrix} t_1, t_2, t_3, \cdots, t_N \end{bmatrix}^T \tag{4.1}$$

$$\mathbf{x} = \begin{bmatrix} x_1, x_2, x_3, \cdots, x_N \end{bmatrix}^T \tag{4.2}$$

Here, $\mathbf{t}$ is a column vector in $\mathbb{R}^{N\times 1}$ representing the target values, and $\mathbf{x}$ is a column vector in $\mathbb{R}^{N\times 1}$ representing the observations. The polynomial equation to fit the data is y(x, w) where

$$y(x,w) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{k=0}^{M} w_k x^k \tag{4.3}$$

In this thesis, M denotes the degree of the polynomial, and for our considerations, M is set to 3. In the expression $y(x,\mathbf{w})$, the variable x represents the y-coordinates of the centroids for a specific vehicle trajectory. The goal is to fit a polynomial curve to these centroids, and $\mathbf{w}$ represents the vector ($\mathbf{w}$ is a column vector in $\mathbb{R}^{(M+1)\times 1}$) of weights w_i.

$$\mathbf{w} = \begin{bmatrix} w_0, w_1, w_2, \cdots, w_M \end{bmatrix}^T \tag{4.4}$$

The coefficients of the polynomial are found by minimizing a measure of the mismatch between the predicted values of the polynomial, denoted as y(x,w), and the actual target t in the training set. This measure of mismatch is quantified using an error function E(w), and the specific error function employed is the sum of the squares of the differences between the predictions y(x_n,w) for each data point x_n and the corresponding target values t_n. Therefore, the objective is to minimize the following expression with respect to the coefficients w

$$E(w) = \sum_{n=1}^{N} [y(x_n, w) - t_n]^2 \tag{4.5}$$

Substituting the expression of the polynomial fitting function equation (4) in (6), we

obtain the following expression:

$$E(w) = \sum_{n=1}^{N} \left[w_0 + w_1 x_n + w_2 x_n^2 \ldots + w_M x_n^M - t_n \right]^2 = \sum_{n=1}^{N} \left(\sum_{j=0}^{M} x_n^j w_j - t_n \right)^2 \tag{4.6}$$

Then we take the derivatives with respect to w_j to solve the linear system:

$$\frac{\partial E(w)}{\partial w_i} = 0$$

And we obtain:

$$\frac{\partial E(w)}{\partial w_i} = \sum_{n=1}^{N} \left(\sum_{j=0}^{M} x_n^j w_j - t_n \right) * 2 * x_n^i = 2 \left(\sum_{n=1}^{N} \sum_{j=0}^{M} x_n^{i+j} w_j - \sum_{n=1}^{N} t_n x_n^i \right) \tag{4.7}$$

$$= 2 \left(\sum_{j=0}^{M} \left(\sum_{n=1}^{N} x_n^{i+j} \right) w_j - \sum_{n=1}^{N} t_n x_n^i \right) \tag{4.8}$$

Setting this derivative to 0 we get:

$$2 \left(\sum_{j=0}^{M} \left(\sum_{n=1}^{N} x_n^{i+j} \right) w_j - \sum_{n=1}^{N} t_n x_n^i \right) = 0 \tag{4.9}$$

$$\sum_{j=0}^{M} \left(\sum_{n=1}^{N} x_n^{i+j} \right) w_j = \sum_{n=1}^{N} t_n x_n^i \tag{4.10}$$

$$\sum_{j=0}^{M} A_{ij} w_j = T_i \tag{4.11}$$

where

$$A_{ij} = \sum_{n=1}^{N} x_n^{i+j} \tag{4.12}$$

and

$$T_i = \sum_{n=1}^{N} x_n^i t_n \tag{4.13}$$

By expanding equation 4.12 into a matrix form we get a relation, as follows:

$$\begin{bmatrix} A_{00} & \cdots & A_{0M} \\ \vdots & \ddots & \vdots \\ A_{M0} & \cdots & A_{MM} \end{bmatrix} = \mathbf{X} \cdot \mathbf{X}^T \tag{4.14}$$

where

$$\mathbf{X}^T = \begin{bmatrix} x_1^0 & \cdots & x_n^0 \\ \vdots & \ddots & \vdots \\ x_1^M & \cdots & x_n^M \end{bmatrix}$$

By using equation 4.11 and equation 4.14, simplify the cost function, and solve the equation to obtain $\mathbf{w}$ as follows: where $\mathbf{X}^T$ is a matrix with dimensions $\mathbb{R}^{(M+1)\times N}$.

$$\mathbf{X}^T \cdot \mathbf{X} \cdot \mathbf{w} = \mathbf{X}^T \cdot \mathbf{t} \quad \Rightarrow \quad \mathbf{w} = (\mathbf{X}^T \cdot \mathbf{X})^{-1} \cdot \mathbf{X}^T \cdot \mathbf{t} \tag{4.15}$$

For $(\mathbf{X}^T\mathbf{X})^{-1}$ to exist (i.e., be non-singular or invertible), the matrix $\mathbf{X}^T\mathbf{X}$ must be full rank. A matrix is a full rank if and only if its determinant is non-zero. Therefore, if $\det(\mathbf{X}^T\mathbf{X}) \neq 0$, the matrix $(\mathbf{X}^T\mathbf{X})^{-1}$ has an inverse, and the solution $\mathbf{W} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$ can be computed. However, if $\det(\mathbf{X}^T\mathbf{X}) = 0$, the matrix $\mathbf{X}^T\mathbf{X}$ is singular, and its inverse does not exist. In such a case, the polynomial regression problem may not have a unique solution, and regularization techniques may need to be considered as explained by (Vultureanu-Albişi & Bădică 2022). The fitted curves depicting the centroid paths of various vehicles across different video frames are presented in Figure 4.1. In this representation, the red curve signifies the actual centroid points of a vehicle tracked throughout the frames, while the blue curve represents the fitted curve for these centroid points using polynomial fitting. This visualization offers insights into the trajectories of multiple vehicles captured in the video frames. This fitting process aims to approximate the actual trajectory of the vehicle's movement, facilitating smoother visualization and analysis of its path across the frames.

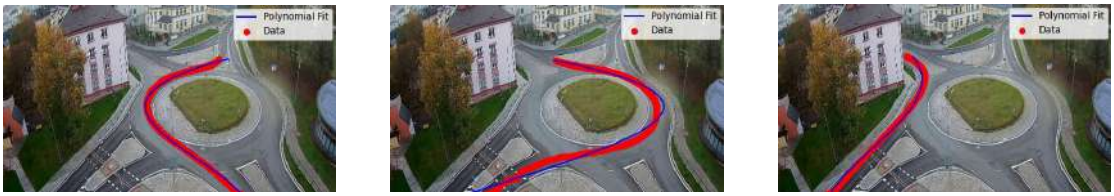


Figure 4.1: Fitted curves(in Blue) along the centroid paths (in Red) of different vehicles.

4.1.3 Sub-trajectory fitting for accurate path modeling

To address the limitations of global polynomial fitting, where a single curve fails to capture complex vehicular motion accurately, we implement a sub-trajectory-based curve fitting approach shown in figure 8. Each vehicle trajectory is segmented into equal-length sub-trajectories. For each sub-trajectory, a second-order polynomial is fitted to model the local path curvature more precisely. This method provides better flexibility in tracking sudden turns and subtle directional changes that occur due to lane shifts or driving behavior.

Given a trajectory $T = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, we divide it into k equal segments, and for each segment, we fit a polynomial:

$$y_i(x) = a_i x^2 + b_i x + c_i, \quad i = 1, 2, \ldots, k$$

These coefficients $\{a_i, b_i, c_i\}$ from all segments are concatenated to form the feature vector representing the trajectory. For $k = 20$ and a second-order polynomial, we obtain 60 features per trajectory.



(a) (b) (c)

Figure 4.2: Images depicting curve fitting considering the entire curve as a single trajectory (a), curve fitting using sub-trajectories (b), and the sub-trajectories curve fitting shown in 20 different segments of the trajectory (c).

4.2 Data Generation

4.2.1 Data imbalance in trajectory classes

In our research, we observed a prevalent challenge related to data imbalance among trajectory classes. Specifically, certain trajectory types were represented by a disproportionately large number of data points compared to others. This imbalance can significantly impact model training, as it tends to skew the model's learning process, favoring the more frequently occurring trajectories and overlooking the less common ones. Such a bias not only hampers the model's performance but also increases the risk of overfitting, where it performs well on familiar data but fails to generalize to new or rare patterns. Notably, this is particularly problematic for identifying abnormal trajectories, which, despite being fewer in number, are often the most critical for the classifier to detect accurately. To address this challenge, we employ synthetic data generation. Rather than relying solely on existing data points, we create additional data using a technique known as the Multi-Gaussian approach. This method allowed us to treat each set of polynomial coefficients within a trajectory as unique features, enabling us to generate additional data points that closely resembled the underrepresented trajectories.

Our approach involves fitting a multivariate Gaussian distribution to the x and y coordinate data of all sub-trajectories within each label. Let (x_{ij}, y_{ij}) represent the coordinates of data points in sub-trajectory i within label j. The probability density function (PDF) of a multivariate Gaussian distribution with mean μ_j and covariance matrix Σ_j is given by:

$$f(\mathbf{x}_{ij}, \mathbf{y}_{ij}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}_{ij} - \mu_j)^T \Sigma^{-1} (\mathbf{x}_{ij} - \mu_j)\right) \tag{4.16}$$

where $\left|\Sigma_j\right|$ represents the determinant of the covariance matrix and Σ_j^{-1} denotes the inverse of the covariance matrix. Fitting a multivariate Gaussian distribution to the trajectory data allows us to capture the spatial correlation between the x and y coor-

dinates, effectively modeling the spatial distribution of the data within each trajectory class. This process is repeated for each label, resulting in a set of multivariate Gaussian distributions, one for each trajectory class. These distributions represent the spatial characteristics of the data within each class, enabling us to generate synthetic data points that closely resemble the original trajectory patterns. By following these steps, we can generate synthetic data by capturing the spatial distribution of centroid points for each label.

4.2.2 Data augmentation

To enhance the generalization ability of the LSTM model and ensure robustness in anomaly detection, a substantial dataset of vehicle trajectories is essential. However, due to the limited availability of labeled abnormal trajectories, synthetic data generation and augmentation techniques are employed.

Normal trajectory generation: Existing trajectories were used as templates to generate variations by applying small perturbations to the polynomial coefficients. For example, given a coefficient vector $\mathbf{w} = [a_1, b_1, c_1, \ldots, a_k, b_k, c_k]$ from the sub-trajectory fitting process, new vectors were generated as:

$$\mathbf{w}' = \mathbf{w} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

where ε is Gaussian noise with a small standard deviation σ. This preserves the underlying path pattern while introducing natural variability.

Abnormal trajectory generation: To simulate anomalies, modifications were applied to the original coefficient vectors to deliberately induce abrupt directional changes or inconsistent sub-paths. These include:

- sudden reversal would be too drastic. Better option could have been suddenly switching to another trajectory (crossing the current one) but going in the opposite direction. in certain sub-trajectories.

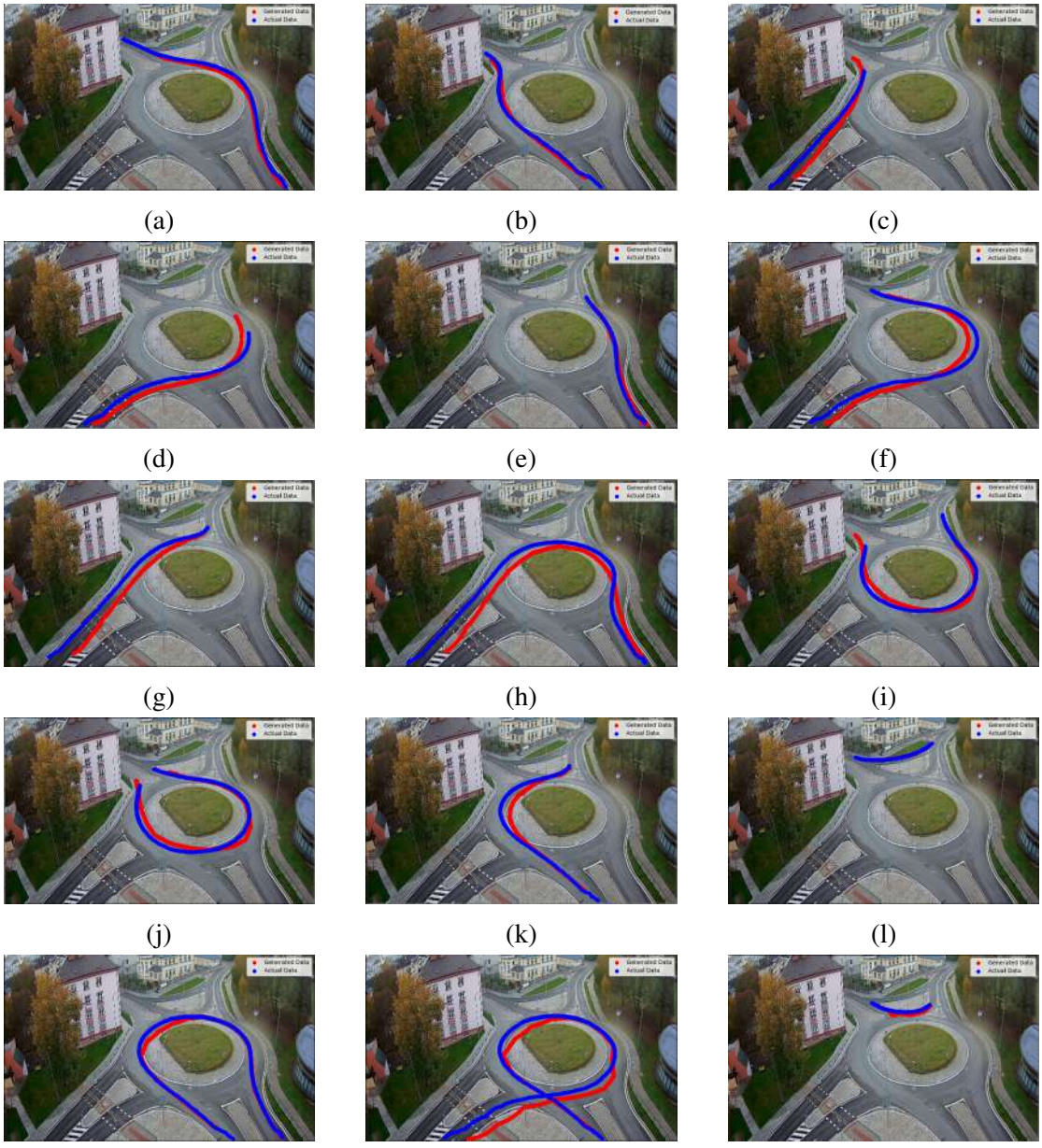
- Injecting spikes by amplifying specific polynomial coefficients.

Such operations emulate behaviors like illegal turns, erratic lane shifts, or sudden stops. Examples of generated abnormal trajectories are illustrated in the thesis and validated to reflect realistic yet diverse anomalous patterns. These synthetic data points were integrated into the training and testing pipeline of the LSTM Autoencoder, leading to improved classification robustness and overall model performance.

The goal of this approach is to fill in the gaps in our dataset, particularly for trajectories with fewer data points. By generating more data that reflects the patterns found in existing trajectories, we aimed to provide the model with a more balanced training set. Figure 4.4 illustrates the effectiveness of our approach in generating synthetic data. By augmenting our dataset in this manner, we aimed to enhance the overall performance and robustness of our model, ultimately leading to more accurate and reliable trajectory analysis across various domains.For this thesis there are 15 normal trajectory classes, and 5000 samples for each trajectory class (after augmentation), totaling to 75,000 samples of normal trajectories and 4,000 samples of abnormal trajectories has been generated. The generated abnormal trajectories are shown in Figure 4.5. Three types of abnormal trajectories have been generated.



Figure 4.3: Normal trajectories of a few vehicles captured in different colors



(a) (b) (c)

(d) (e) (f)

(g) (h) (i)

(j) (k) (l)

(m) (n) (o)

Figure 4.4: Generated normal trajectories (in Blue) compared to the already encountered sample trajectory (in Red).



(a) (b) (c)

Figure 4.5: Generated abnormal trajectories

4.3 Summary

The research presented in the thesis focuses on improving trajectory analysis, particularly in the context of vehicle path tracking. Traditional methods, like linear regression, often struggle to accurately capture the intricate and non-linear nature of vehicle trajectories. To address this limitation, the thesis introduces polynomial fitting and sub-trajectory fitting techniques. Polynomial regression offers a more flexible approach by fitting a curve to the data, allowing for better representation of non-linear relationships. By employing polynomial curves of different degrees, the method can capture the complex patterns observed in vehicle movements more effectively.

Moreover, the thesis proposes a novel sub-trajectory fitting approach, where the entire trajectory is divided into smaller segments, and polynomial fitting is applied to each segment individually. This method enables a more granular analysis of the trajectory, capturing finer details and variations in the vehicle's path. Additionally, the thesis explores the challenges of data imbalance in trajectory datasets and introduces synthetic data generation techniques to address this issue. By generating additional data points using the Multi-Gaussian approach, (This observation resonates with findings in (Coşar et al. 2017), where the distribution of speed in a given zone is described as following a Gaussian distribution.) the dataset becomes more balanced, leading to improved model performance and generalization.

Overall, the research contributes to advancing trajectory analysis techniques, offering more accurate and robust methods for understanding vehicle movement patterns. The combination of polynomial fitting, sub-trajectory fitting, and synthetic data generation provides a comprehensive framework for analyzing complex trajectories, with potential applications in various domains beyond vehicle tracking.

5 Trajectory Classification using LSTMs

5.1 LSTM-Based Classifier Model

Long Short-Term Memory (LSTM) networks are an advanced form of recurrent neural networks (RNNs), specifically engineered to overcome the limitations posed by vanishing or exploding gradients during backpropagation, especially in tasks involving long-range dependencies. The architecture of LSTM incorporates memory cells and gating mechanisms, which enable the model to selectively retain or discard information across time steps, ensuring the preservation of relevant features throughout the sequence.

An LSTM unit comprises three primary gates: the input gate, forget gate, and output gate. These gates regulate the information flow within the memory cell, allowing the network to decide what information to store, update, or output.

Input Gate

The input gate controls the extent to which new information should influence the cell state. It is defined by the sigmoid activation applied to a weighted sum of the current input x_t, the previous hidden state h_{t-1}, and a bias term:

$$i_t = \sigma(W_i x_t + W_i h_{t-1} + b_i) \tag{5.1}$$

In this equation:

- $W_i \in \mathbb{R}^{N \times N}$ is the input gate weight matrix.

- $x_t \in \mathbb{R}^{N\times 1}$ is the input at time step t.
- $h_{t-1} \in \mathbb{R}^{N\times 1}$ is the previous hidden state.
- $b_i \in \mathbb{R}^{N\times 1}$ is the bias vector.

Forget Gate

The forget gate determines whether the information from the previous cell state c_{t-1} should be retained or discarded. It is computed as:

$$f_t = \sigma(W_f x_t + W_f h_{t-1} + b_f) \tag{5.2}$$

Where:

- $W_f \in \mathbb{R}^{N\times N}$ is the forget gate weight matrix.
- $b_f \in \mathbb{R}^{N\times 1}$ is the forget gate bias.

Candidate Cell State

The candidate cell state $\tilde{c}_t$ introduces new information that may be added to the current memory cell. It is computed as:

$$\tilde{c}_t = \tanh(W_c x_t + W_c h_{t-1} + b_c) \tag{5.3}$$

Where:

- $W_c \in \mathbb{R}^{N\times N}$ is the candidate state weight matrix.
- $b_c \in \mathbb{R}^{N\times 1}$ is the candidate state bias.

Cell State Update

The updated cell state c_t combines the retained previous cell state and the new candidate state, weighted by the forget and input gates respectively:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{5.4}$$

Here, $\odot$ denotes element-wise (Hadamard) multiplication.

Output Gate

The output gate governs which parts of the cell state are output as the hidden state:

$$o_t = \sigma(W_o x_t + W_o h_{t-1} + b_o) \tag{5.5}$$

Where:

- $W_o \in \mathbb{R}^{N \times N}$ is the output gate weight matrix.
- $b_o \in \mathbb{R}^{N \times 1}$ is the output gate bias.

Hidden State Output

Finally, the hidden state h_t is computed by applying a non-linear transformation to the updated cell state and modulating it with the output gate:

$$h_t = o_t \odot \tanh(c_t) \tag{5.6}$$

Each component of the LSTM model operates through matrix multiplications, bias additions, and non-linear activations (sigmoid or tanh), enabling the effective modeling of complex temporal patterns in sequential data.

5.2 LSTM Auto-Encoder

An LSTM Auto-Encoder consists of two main components: the encoder and the decoder. The encoder transforms the input sequence x_t into a compressed latent representation e_t, also referred to as the feature code. This transformation is typically a

deterministic mapping that involves an affine transformation followed by a non-linear activation function, as shown below:

$$e_t = \sigma(W_e x_t + b_e) \tag{5.7}$$

Here, $W_e \in \mathbb{R}^{M \times N}$ is the weight matrix that maps the input $x_t \in \mathbb{R}^{N \times 1}$ to the hidden representation $e_t \in \mathbb{R}^{M \times 1}$, and $b_e \in \mathbb{R}^{M \times 1}$ is the bias vector.

The decoder takes the encoded representation e_t and reconstructs the original input sequence $\hat{x}_t$. This can be expressed as:

$$\hat{x}_t = \sigma(W_d e_t + b_d) \tag{5.8}$$

In this equation, $W_d \in \mathbb{R}^{N \times M}$ is the weight matrix that maps the encoded vector back to the reconstructed input $\hat{x}_t \in \mathbb{R}^{N \times 1}$, and $b_d \in \mathbb{R}^{N \times 1}$ is the corresponding bias vector.

The model is trained to minimize the reconstruction error between the original input and the reconstructed output. This is quantified by a cost function J, defined as the average reconstruction loss across all input samples:

$$J = \frac{1}{p} \sum_{i=1}^{p} L[x_t, \hat{x}_t] \tag{5.9}$$

Here, p denotes the number of input samples, and $L[x_t, \hat{x}_t]$ is the loss function that measures the reconstruction error. In this study, the mean squared error (MSE) is used:

$$L[x_t, \hat{x}_t] = \|x_t - \hat{x}_t\|_2^2 \tag{5.10}$$

For classification tasks, especially in multi-class scenarios, the cross-entropy loss function is employed:

$$\text{Cross-Entropy} = -\sum_{i=1}^{N} y_i \log(p_i) \tag{5.11}$$

Where:

N : Total number of classes

y_i : True probability of class i (typically one-hot encoded)

p_i : Predicted probability for class i

This function evaluates how well the predicted probabilities match the actual class labels. By incorporating LSTM layers into the auto-encoder architecture, the model is able to capture temporal dependencies and dynamic patterns in sequential input data. Both the encoder and decoder utilize LSTM units, making this architecture well-suited for time-series and trajectory data.

5.2.1 Encoding layer

The encoder consists of three sequential LSTM layers, each designed to progressively compress the input trajectory data and extract meaningful temporal features.

- The first LSTM layer processes the input trajectory tensor and uses a hidden state dimension of 256 units.
- The second LSTM layer takes the output of the first and reduces the dimensionality using 128 hidden units.
- The third LSTM layer further compresses the representation with 64 hidden units.

The encoder's forward function passes the input trajectory data through these three LSTM layers, resulting in a compact representation that retains essential temporal characteristics.

5.2.2 Decoding layer

The decoder mirrors the encoder's structure and consists of three LSTM layers followed by a linear layer that reconstructs the input trajectory:

- The first LSTM layer uses 64 hidden units to start decoding from the latent representation.
- The second LSTM layer expands the output with 128 hidden units.
- The third LSTM layer further expands the data with 256 hidden units.
- The final linear layer converts the tensor back into the original trajectory format.

The compressed representation produced by the encoder serves as the latent space, where essential information is preserved and redundant details are removed. This low-dimensional encoding is used by the decoder to reconstruct the input while maintaining the temporal structure of the data.

The overall architecture of the LSTM Auto-Encoder is illustrated in Figure 5.1.

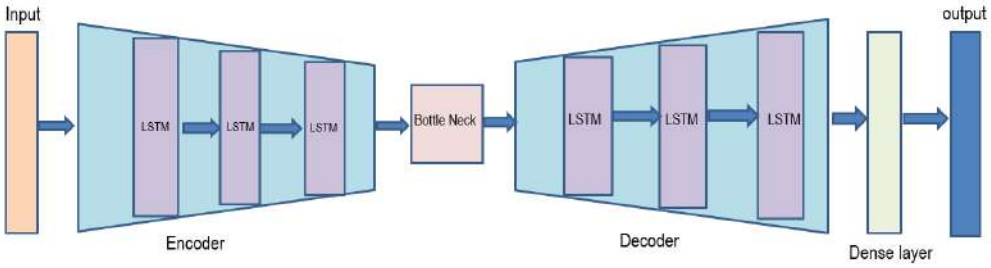


Figure 5.1: Architecture of LSTM-based Auto-Encoder and Decoder

5.3 Trajectory classification

5.3.1 Training with normal and partial-length trajectories

In the initial phase of training, the model was exclusively trained using normal trajectory data. Each trajectory was categorized into one of 15 predefined classes, representing different common vehicle movement patterns observed in the video. The training was performed over 50 epochs, with a learning rate of 0.01, utilizing 70% of the total available data for training. The training dataset consisted of complete trajectories, ensuring the model learned to recognize typical vehicular behavior.

In an alternate training methodology, only 70% of the length of each trajectory was used during training, even for normal class data. This partial-length approach was applied across all 15 trajectory categories to allow the model to learn from incomplete sequences, simulating scenarios where only partial trajectory data might be available.

5.3.2 Testing with full-length and abnormal trajectories

For testing, the remaining 30% of the dataset, which included both normal and abnormal trajectories, was used. The abnormal trajectories were labeled uniquely (as label 16) to differentiate them from the 15 normal trajectory classes. This phase aimed to evaluate the model's generalization ability, particularly its capacity to detect and classify abnormal trajectories that were unseen during training.

In the case of the alternate training methodology, the model was evaluated using full-length trajectories (100%) during testing. This setup aimed to assess the model's performance when handling complete and potentially anomalous sequences, despite being trained on partial trajectory data.

5.3.3 Trajectory classification using LSTM-based autoencoder

For the task of trajectory classification and anomaly detection, an LSTM-based autoencoder was employed. The model was trained to learn the normal vehicular motion patterns by using trajectory data represented through polynomial sub-trajectory coefficients. Each trajectory was divided into 20 sub-segments, with each segment being modeled by a second-order polynomial, yielding a 60-dimensional feature vector.

The autoencoder works by encoding the 60-dimensional input into a compact latent representation, followed by decoding it to reconstruct the original trajectory. The effectiveness of the reconstruction is measured using the Mean Squared Error (MSE), which quantifies the difference between the original and reconstructed feature vectors. The reconstruction loss, as defined below, serves as a key metric for identifying deviations from normal behavior:

$$\text{Reconstruction Loss} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$$

where:

- x_i represents the i-th element of the original sub-trajectory coefficient vector,
- $\hat{x}_i$ denotes the reconstructed value for the i-th element,
- $N = 60$ is the dimensionality of the input feature vector.

5.3.4 Model evaluation and abnormality detection

During the testing phase, the trained autoencoder was used to evaluate new trajectory data. If the reconstruction loss for a test trajectory exceeded a predefined threshold—determined through statistical analysis of training loss values—the trajectory was classified as abnormal. This threshold was chosen to ensure that only those trajectories that significantly deviate from normal patterns were flagged as anomalies.

The model's performance was evaluated using a confusion matrix, resulting in a classification accuracy of approximately 98.86%. This high accuracy indicates that the model, based on the sub-trajectory learning approach, effectively detects abnormal trajectories while maintaining robustness in real-world scenarios.

5.4 Results and Observations

Table 5.1 compares the performance metrics obtained when using the full length of trajectories versus using only 70% of the trajectory length during training. When the model was trained on the full length of each trajectory, it achieved an accuracy of 98.86%, indicating that it correctly classified nearly all trajectories. The precision, recall, and F1 score were also high, approximately 0.99, reflecting a strong balance between correctly identifying true positive and negative instances. When only 70% of each trajectory was used during training, the accuracy decreased slightly to 96.28%. Nonetheless, the precision, recall, and F1 score remained high at around 0.9766, 0.9628,

and 0.965, respectively. These results suggest that the model maintained reliable performance, even when trained on truncated trajectory data. Visual representations of trajectory samples, as shown in Figures 5.2c and 5.2d, illustrate instances where normal trajectories exhibit partial similarity to abnormal ones. For example, some red curves, which represent normal trajectories, share segments with paths of abnormal trajectories. This overlap suggests that certain portions of otherwise normal paths can mimic abnormal patterns. Such partial similarities can lead to occasional misclassifications, where the model identifies normal trajectories as abnormal due to shared path characteristics. These observations highlight the inherent difficulty in distinguishing between normal and abnormal behavior when their movement patterns partially overlap.

Accuracy

Accuracy measures the proportion of correctly classified instances among the total instances in the dataset.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

Precision

Precision measures the proportion of correctly predicted positive cases among all cases predicted as positive.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Recall

Recall measures the proportion of correctly predicted positive cases among all actual positive cases.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

F1 Score

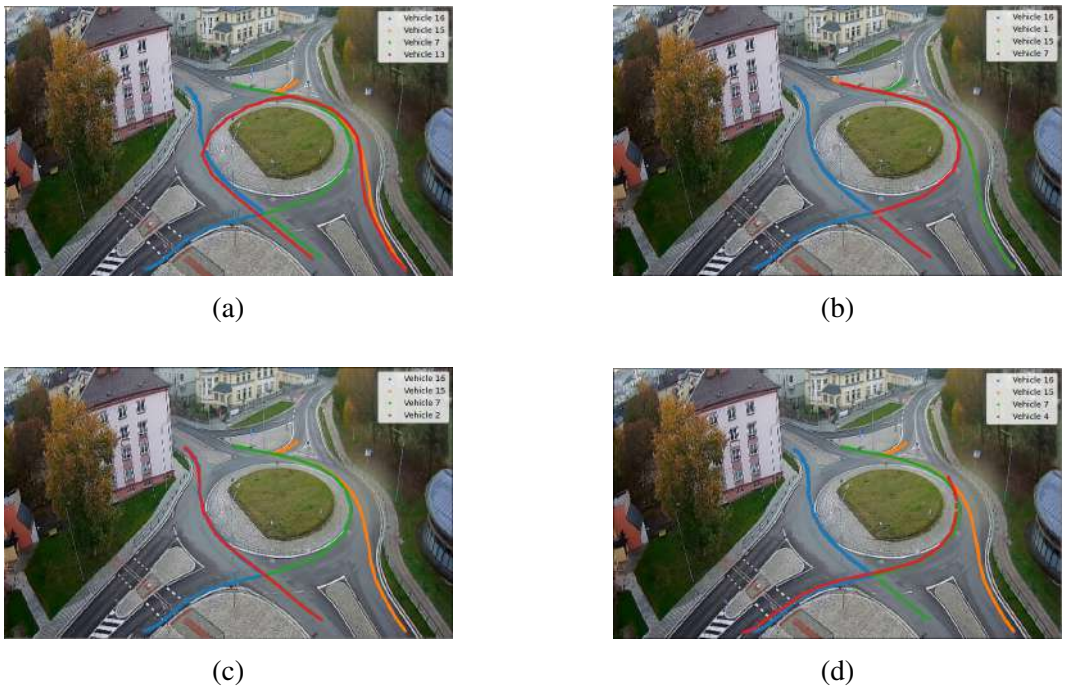
F1 score is the harmonic mean of precision and recall, providing a balance between the two metrics.

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Considering full trajectory	
Metric	Value
Accuracy	98.86%
Precision	0.99
Recall	0.988
F1 Score	0.988

Considering 70% length of trajectory	
Metric	Value
Accuracy	96.28%
Precision	0.9766
Recall	0.9628
F1 Score	0.965

Table 5.1: Comparison of metrics for full length and 70% length of trajectories



(a) (b)

(c) (d)

Figure 5.2: In Figure 5.2b, the orange trajectory represents a normal trajectory with a path that overlaps a known abnormal path. Similarly, in Figure 5.2a, the red normal trajectory shows considerable overlap with abnormal trajectories.

5.5 Summary

This study explored the use of an LSTM-based autoencoder for identifying abnormal trajectories from a collection of vehicle motion patterns. The model's training and

evaluation revealed several important insights:

- **Training with normal trajectories:** The autoencoder was initially trained using only trajectories labeled as normal. The resulting model exhibited a high classification accuracy of 98.86% during testing, demonstrating its ability to effectively learn and distinguish between regular and irregular patterns in trajectory data.

- **Reduced-length training strategy:** An alternate strategy involved training the model on only 70% of the trajectory length while testing it on complete trajectories. Even with this partial input, the model achieved an accuracy of 96.28%, showing that it retained strong predictive capabilities despite having access to less data during training.

- **Evaluation metrics:** Across both training scenarios, the model produced consistently strong values for precision, recall, and F1 score. This indicates that it maintained a good balance between identifying true positives and avoiding false classifications, regardless of the amount of training data.

- **Trajectory overlap and classification errors:** In some cases, normal trajectories were incorrectly identified as abnormal. These errors occurred when segments of normal paths closely resembled those typically seen in abnormal patterns. This overlap highlights a common challenge in trajectory analysis, where similar-looking movements can lead to ambiguity in classification.

The results emphasize the model's capacity to generalize well to unseen data, even when trained under constrained conditions. Its strength lies in its ability to autonomously extract informative features from trajectory sequences, enabling it to perform robustly in scenarios involving noisy or incomplete inputs. These characteristics make the LSTM autoencoder a promising tool for real-world applications in traffic surveillance and behavior analysis. Moving forward, refining the model to better handle edge cases and ambiguous trajectories could further enhance its reliability and performance in practical deployments.

6 Vehicle Speed-Based Classification

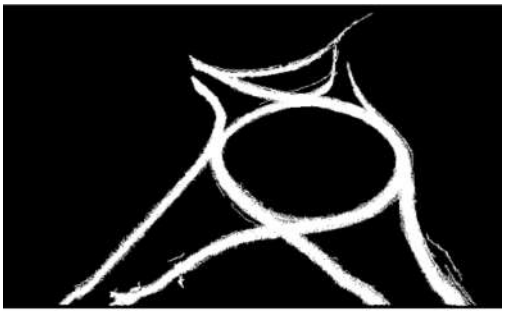
6.1 Introduction

While trajectory-based analysis provides significant insights into vehicle movement patterns, speed is a critical dimension for understanding traffic behavior and detecting anomalies. This chapter introduces a method for classifying abnormal vehicle motion based on speed profiles computed from trajectory data. The analysis is designed to be adaptive and fine-grained by using a pixel-level speed mapping strategy rather than restricting evaluation to predefined regions.

6.2 Pixel-wise Speed Profiling Strategy

6.2.1 Trajectory pixel mapping

Instead of analyzing speed over a static Region of Interest (ROI), our method considers every pixel that lies along any vehicle trajectory across the video frame. These pixels are dynamically identified during vehicle tracking, and a binary mask is generated to mark their presence. Figure 6.1a shows this binary mask, which includes all trajectory pixels.Figure 6.2 illustrates an example speed distribution obtained from a local 15-pixel neighborhood. Such distributions form the basis for computing adaptive thresholds used in anomaly detection.



(a) Trajectory Pixel Mask



(b) 15-pixel Neighborhood Region

Figure 6.1: (a) Binary mask of pixels traversed by vehicles. (b) A local 15-pixel neighborhood used to aggregate speed information around trajectory points.

6.2.2 Neighborhood-based speed aggregation

To capture the localized motion behavior, a 15-pixel radius neighborhood is considered around each trajectory pixel. This allows us to calculate a representative distribution of vehicle speeds within local spatial contexts, as shown in Figure 6.1b. This strategy supports adaptive anomaly detection, especially in diverse and densely occupied areas of the frame. Figure 6.2 illustrates an example speed distribution obtained from a local 15-pixel neighborhood. Such distributions form the basis for computing adaptive thresholds used in anomaly detection.

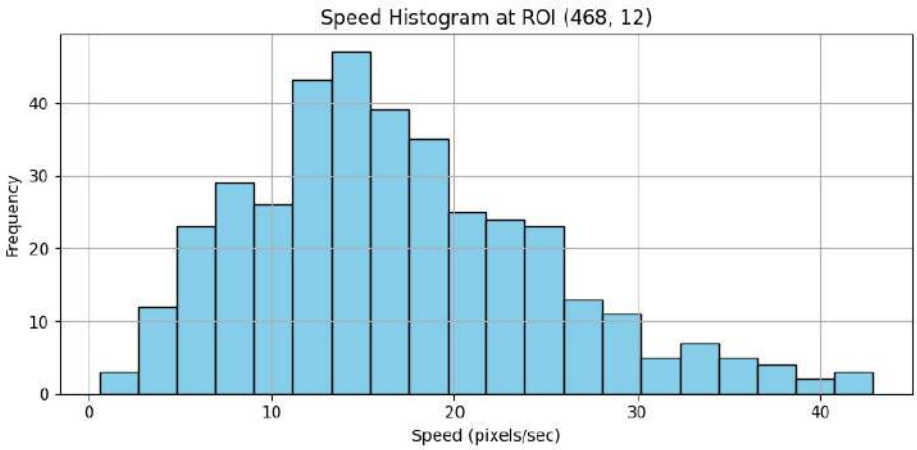


Figure 6.2: Speed distribution aggregated over the 15-pixel neighborhood around multiple trajectory pixels. The horizontal axis indicates speed values, while the vertical axis indicates frequency. Peaks reflect commonly observed speeds in that local region.

6.3 Vehicle Speed Calculation

6.3.1 Centroid-based speed estimation

Vehicle speed is computed by tracking the movement of the vehicle centroid across frames. For two consecutive positions (x_1, y_1) and (x_2, y_2) observed n frames apart, the speed v is estimated as:

$$v = \frac{d}{\Delta t} = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{n \cdot \Delta t}$$

where:

- (x_1, y_1): Centroid coordinates when the vehicle enters the region,
- (x_2, y_2): Centroid coordinates when the vehicle exits the region,
- n: Number of frames between the two positions,
- Δt: Time per frame (inverse of the video frame rate).

This formula ensures consistency in measuring speeds across varying frame intervals and trajectories.

6.4 Speed-Based Anomaly Detection

6.4.1 Statistical thresholding

Once the speed data is gathered over all valid trajectory segments, a statistical analysis is performed. Let μ and σ denote the mean and standard deviation of the collected speeds in a region. A trajectory is flagged as abnormal if its corresponding vehicle speed lies outside the 66.7% confidence interval:

$$\text{Normal Speed Range} = [\mu - \sigma, \mu + \sigma]$$

This approach helps in identifying vehicles that are moving significantly faster or slower than the majority of traffic within the local context.

6.4.2 Advantages of local speed thresholding

This localized, adaptive detection framework has several benefits:

- **Robustness to heterogeneous traffic flow:** Speeds are analyzed relative to their spatial context, enabling better differentiation in complex scenes.
- **Detection of slow-moving or overspeeding vehicles:** Both extremes of motion behavior are captured without predefined speed limits.
- **Enhanced anomaly detection:** When combined with trajectory-based methods, speed-based filtering improves the detection of suspicious or unexpected motion patterns.

6.5 Summary

In summary, the speed-based classification approach complements trajectory analysis by introducing a quantitative metric for motion evaluation. The pixel-level granularity, neighborhood-based speed profiling, and statistical thresholding make this approach versatile and reliable. It plays a vital role in identifying abnormal vehicle behavior, especially in environments where trajectory overlaps or ambiguous paths can lead to classification errors. Future extensions could involve learning speed thresholds adaptively using clustering or deep learning to further enhance detection precision.

7 Conclusion

7.1 Summary of the Work

This study presents an integrated framework for vehicle detection, segmentation, trajectory modeling, and anomaly detection in traffic videos. YOLOv8 and Mask R-CNN were used for accurate detection and segmentation under complex conditions. Centroid-based tracking, enhanced via image moments and Euclidean distance, ensured ID consistency across frames. To model motion effectively, trajectories were segmented into sub-trajectories using polynomial fitting. A dataset of 1300 real and 79,000 synthetic trajectories (75,000 normal, 4,000 abnormal) was prepared. An LSTM-based autoencoder, trained solely on normal data, successfully detected abnormalities by learning normal behavior patterns. Additionally, a region-of-interest (ROI) based speed analysis detected anomalous speeds using statistical thresholds. The system demonstrates strong potential for traffic surveillance and safety applications by combining spatial and temporal cues.

7.2 Future Work

Future efforts will focus on improving tracking precision and predictive capabilities:

- **Direction-aware path prediction:** Extend trajectory analysis to incorporate vehicle movement direction for predicting future paths and detecting abnormal deviations.

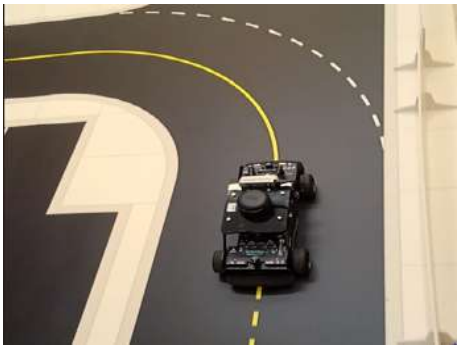
- **Real-time deployment and dataset utilization on Quanser car** Real-time deployment of the proposed framework using the Quanser car(Q-Car) platform (setup illustrated in Figure 7.1, a research-grade autonomous vehicle equipped with four directional cameras and depth sensors for 360-degree perception. During preliminary experiments, both normal and abnormal vehicle trajectories were recorded and manually annotated. This dataset, capturing diverse behaviors such as smooth navigation and erratic movement, will be instrumental for validating and refining the anomaly detection system.



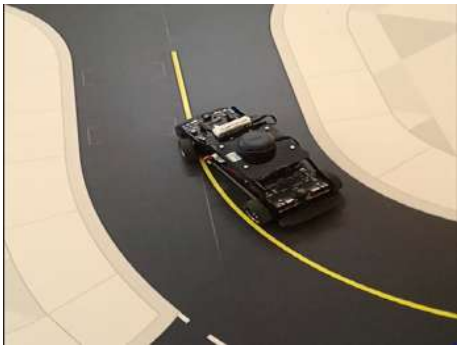
(a) Q-Car hardware



(b) Path planning and trace



(c) Motion execution snapshot



(d) Motion execution snapshot

Figure 7.1: Q-Car setup

.

These enhancements aim to improve reliability and enable real-time deployment in intelligent traffic systems.

Publications

List of Publication

Conference

K. Konapalli, Y. Peddapothula and N. Ghosh, "Traffic Vehicle Tracking and Trajectory Classification Using LSTM," 2024 IEEE 21st India Council International Conference (INDICON), Kharagpur, India, 2024, pp. 1-6, doi: 10.1109/INDICON63790.2024.10958278.

Bibliography

Afdhal, A., Saddami, K., Sugiarto, S., Fuadi, Z. & Nasaruddin, N. (2023), Real-time object detection performance of yolov8 models for self-driving cars in a mixed traffic environment, *in* '2023 2nd International conference on computer system, information technology, and electrical engineering (COSITE)', IEEE, pp. 260–265.

Bawankule, R., Gaikwad, V., Kulkarni, I., Kulkarni, S., Jadhav, A. & Ranjan, N. (2023), Visual detection of waste using yolov8, *in* '2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS)', pp. 869–873.

Coşar, S., Donatiello, G., Bogorny, V., Garate, C., Alvares, L. O. & Brémond, F. (2017), 'Toward abnormal trajectory and event detection in video surveillance', *IEEE Transactions on Circuits and Systems for Video Technology* **27**(3), 683–695.

Hussain, S. M. F., Hamza, S. M. & Samad, A. (2022), Image segmentation for autonomous driving using u-net inception, *in* '2022 7th International Conference on Signal and Image Processing (ICSIP)', pp. 426–429.

Leilei, D., Zheng-liang, S., Jin-gang, G. & Hong-tong, Q. (2012), Study on traffic organization and management strategies for large special events, *in* '2012 International Conference on System Science and Engineering (ICSSE)', pp. 432–436.

Morris, B. T. & Trivedi, M. M. (2008), 'Learning, modeling, and classification of vehicle track patterns from live video', *IEEE Transactions on Intelligent Transportation Systems* **9**(3), 425–437.

Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. (2016), You only look once: Unified, real-time object detection, *in* '2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)', pp. 779–788.

Sakpal, N. S. & Sabnis, M. (2018), Adaptive background subtraction in images, *in* '2018 International Conference On Advances in Communication and Computing Technology (ICACCT)', pp. 439–444.

Tiwari, A., Singh, P. K. & Amin, S. (2016), A survey on shadow detection and removal in images and video sequences, *in* '2016 6th International Conference - Cloud System and Big Data Engineering (Confluence)', pp. 518–523.

Vultureanu-Albişi, A. & Bădică, C. (2022), The model of regularization coefficient in polynomial regression for modelling the spread of covid-19 in romania, *in* '2022 23rd International Carpathian Control Conference (ICCC)', pp. 94–100.

Wu, M., Yue, H., Wang, J., Huang, Y., Liu, M., Jiang, Y., Ke, C. & Zeng, C. (2020), 'Object detection based on rgc mask r-cnn', *IET Image Processing* **14**(8), 1502–1508.

Zeng, K., Zhuang, X., Xie, Y. & Xi, Z. (2021), Hypersonic vehicle trajectory classification using improved cnn-lstm model, *in* '2021 IEEE International Conference on Unmanned Systems (ICUS)', pp. 691–696.

Curriculum Vitae

Konapalli Kumar

Kondapalem, Visakhapatnam, Andhra Pradesh, India, Pin-531026

Research Area

- Image Processing, Computer Vision, Machine Learning, and Deep Learning.

Education

M.Tech Department of Electrical Engineering — August 2023 - May 2025
Indian Institute of Technology Kharagpur, India
Major: Signal Processing and Machine Learning

B.Tech RGUKT, Nuzvid, India — August 2015 - May 2019
Major: ECE
Thesis: Pattern Recognition Techniques for Object Identification
Percentage: 87.9%